†

ETHIOPIAN ORTHODOX TEWAHEDO CHURCH

የኢትዮጵያ ኦርቶዶክስ ተዋሕዶ ቤተ ክርስቲያን ሰንበት ትምህርት ቤቶች አንድነት

LEARNING PLATFORM

USER & DEPLOYMENT MANUAL

A comprehensive guide to authoring curriculum, managing users, offline-first student learning, and enterprise production DevOps deployment across parish environments.

Production MVP 1.0 | Offline-First PWA & Mobile | Docker & Cloud Ready

Document ID:	EOTC-OPS-MAN-2026-V1	**Classification:**	Parish & DevOps Operations
Release Version:	1.0.0 (Production Verified)	**Release Date:**	October 2026
Client Stacks:	Next.js 15 PWA · React Native Expo	**Server Stacks:**	Express.js · Prisma · PostgreSQL 16
Offline Engine:	IndexedDB (Web) · SQLite (Mobile)	**Asset Delivery:**	Cloudinary CDN + Local Cached Blobs

Table of Contents

1. System Architecture & Core Concepts Page 3

- 1.1 System Mission & Architecture Overview
- 1.2 Monorepo Project Structure
- 1.3 The Transactional Learning Package Model
- 1.4 Security & Role-Based Access Control (RBAC)

2. Administrator Manual ("How to Use — Admin") Page 5

- 2.1 Accessing the Admin Management Portal
- 2.2 User & Parish Role Administration
- 2.3 Curriculum Hierarchy Management (Grades, Subjects, Chapters)
- 2.4 Textbook Document & Clamped Page Range Mapping
- 2.5 Authoring Practice Quizzes, Questions & Bilingual Choices
- 2.6 Attaching Approved Reference Documents & Links
- 2.7 Atomic Package Publishing & Integrity Checksums

3. Student Manual ("How to Use — Student") Page 9

- 3.1 Web PWA & Mobile App Access and Installation
- 3.2 Offline Authentication Credential Caching
- 3.3 Curriculum Discovery & Download Manager
- 3.4 Offline Study Experience (Clamped PDF Reader & References)
- 3.5 Taking Practice Quizzes in Airplane Mode
- 3.6 Automatic Background Progress & Attempt Synchronization
- 3.7 Storage Management & Cleanup Controls

4. DevOps & Production Deployment Guide ("How to Deploy") Page 13

- 4.1 Infrastructure Requirements & Hardware Sizing
- 4.2 Environment Variables & Cryptographic Secrets
- 4.3 Database Setup, Migrations & Automated Seeding
- 4.4 Containerized Deployment (Docker Compose + Nginx Gateway)
- 4.5 Host-Level Service Deployment (Systemd + PM2 + Node.js)
- 4.6 Mobile Application Build & Parish Offline Sideloading (EAS / APK)
- 4.7 Automated Database Backup & Disaster Recovery Scripts
- 4.8 Live Production Smoke Testing & Health Monitoring

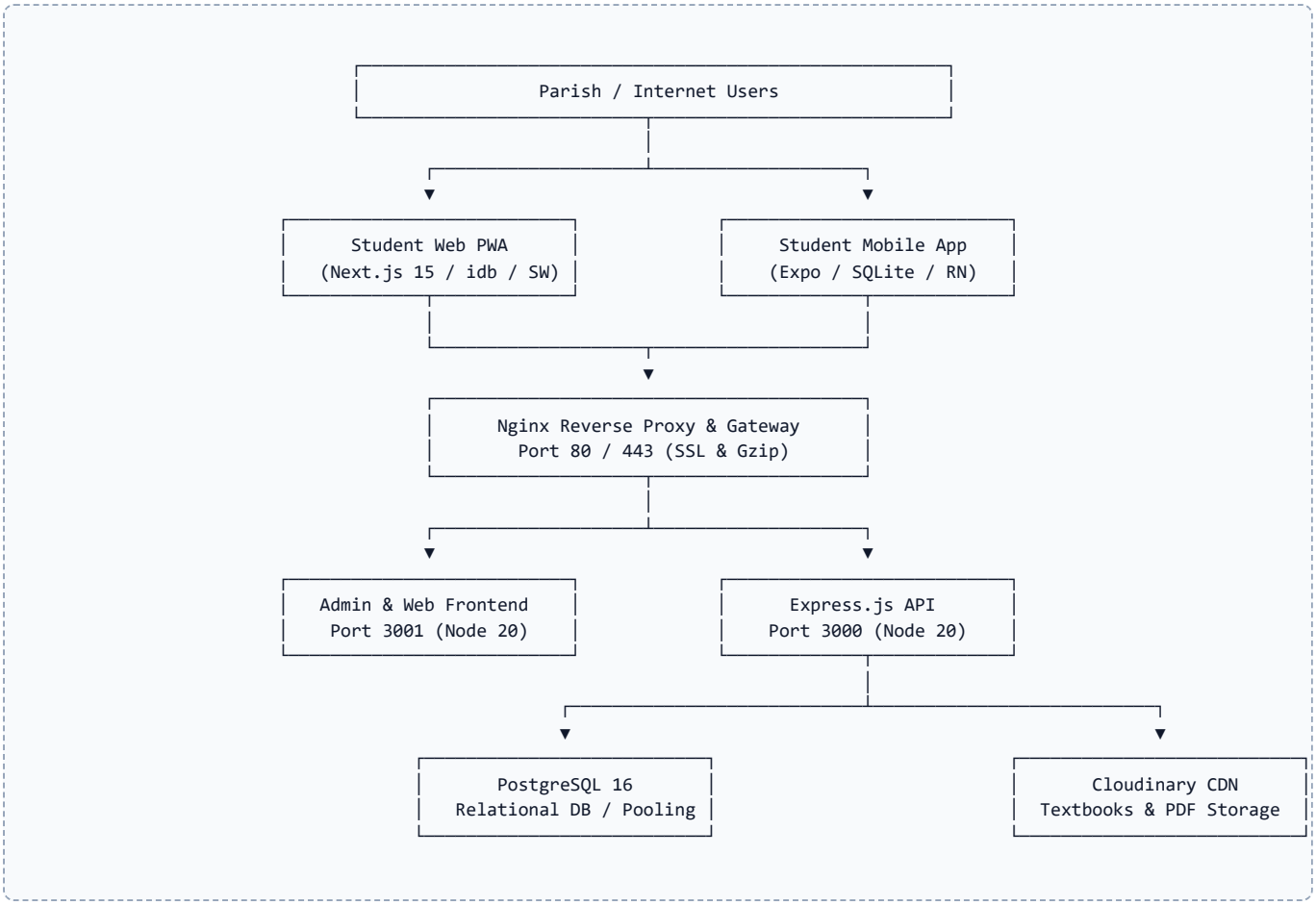
5. Appendices & Reference Sheets Page 19

- 5.1 CLI Operations Cheat Sheet
- 5.2 API Error Code Reference Matrix
- 5.3 Parish Offline Deployment Scenarios (Rural Parishes & Labs)

1. System Architecture & Core Concepts

1.1 System Mission & Architecture Overview

The **EOTC Senbet Tmhrtbet Learning Platform** is engineered specifically for the educational context of the Ethiopian Orthodox Tewahedo Church. In many urban and rural parishes across Ethiopia and the diaspora, continuous high-speed internet connectivity cannot be guaranteed. The platform operates on a strict **offline-first paradigm**: once an approved learning package is downloaded, a student can study textbooks, take practice quizzes with instant local scoring, and consult approved reference materials entirely offline without network access.



1.2 Monorepo Project Structure

The codebase is organized as a high-performance `pnpm` workspace ensuring strict contract sharing, automated typecheck verification, and code reuse across web, mobile, and server environments:

PACKAGE / APP	TECHNOLOGY	DESCRIPTION & RESPONSIBILITIES
`packages/shared`	TypeScript, Zod	Canonical Zod validation schemas, API contracts, RBAC enums, and manifest types shared across API, Web, and Mobile.
`apps/api`	Express, Prisma, PostgreSQL	RESTful API engine providing authentication, user management, curriculum publishing, download packaging, and idempotent sync.

PACKAGE / APP	TECHNOLOGY	DESCRIPTION & RESPONSIBILITIES
`apps/web`	Next.js 15, Tailwind, `idb`	Responsive Student Web PWA and Administrator Management Portal with client-side IndexedDB offline storage and Service Worker caching.
`apps/mobile`	Expo SDK 52, `expo-sqlite`	Native cross-platform mobile application for Android and iOS utilizing SQLite repositories and local file persistence.

1.3 The Transactional Learning Package Model

The core architectural innovation is the **Learning Package**: an atomic, versioned, and self-contained unit representing a single curriculum chapter. A package is only declared `AVAILABLE_OFFLINE` when the 5-step transactional verification pipeline succeeds:

> **The 5-Step Download Completion Invariant**
>
> 1. **Manifest Retrieval**: Fetches the canonical JSON manifest containing chapter metadata, checksums, and version.
> 2. **Textbook PDF Download & Verification**: Downloads the textbook document and verifies non-zero file size and SHA-256 checksum.
> 3. **Downloadable Reference Verification**: Downloads all attached reference files marked `isDownloadable: true`.
> 4. **Quiz Storage**: Commits quiz questions, bilingual choices, and local answer keys into the local client database.
> 5. **Atomic Manifest Commit**: Atomically persists the package manifest with status `AVAILABLE_OFFLINE`. If any resource fails, status remains `FAILED` or `PARTIALLY_DOWNLOADED`.

1.4 Security & Role-Based Access Control (RBAC)

The platform enforces strict role separation across three primary user tiers:

- **ADMIN**: Sole curriculum author and publisher; manages parish accounts; accesses `/api/admin/*` and `/admin`.
- **TEACHER**: Authorized church educators; can view student progress and curricula; blocked from administrative mutations.
- **STUDENT**: Learner accounts; browses approved curricula, downloads learning packages, and syncs quiz attempts.

Authentication relies on dual-token architecture: short-lived **15-minute JWT access tokens** and **30-day revocable refresh tokens** with cryptographic rotation. If an account is deactivated by an administrator, the authentication middleware immediately rejects existing tokens with `403 ACCOUNT_INACTIVE`.

2. Administrator Manual ("How to Use — Admin")

2.1 Accessing the Admin Management Portal

Administrators manage church education through the centralized web studio built into `apps/web`:

1. Navigate your browser to `http://localhost:3001/login` (or `https://learn.senbet.eotc-andnet.org/login`).
2. Enter your administrator email and password (initial seeded credentials: `admin@eotc.org` / `AdminPass123!`).
3. Upon authentication, users with the `ADMIN` role are automatically redirected to the administrative studio at `/admin`.

The administrative navigation bar provides instant access to **Dashboard**, **User Management**, **Curriculum Studio**, and **Parish Analytics**.

2.2 User & Parish Role Administration

Located at `/admin/users`, the user management module allows administrators to onboard church educators, enroll students, and control account statuses:

Creating a New User Account

1. Click the gold **"Add User"** button in the upper right corner.
2. Fill in user details:
   - **Full Name** (e.g., *Dawit Hailu / ዳዊት ኃይሉ*).
   - **Email Address** (unique identifier used for login).
   - **Password** (minimum 8 characters, containing uppercase, lowercase, and numbers).
   - **Role Selection**: Select `STUDENT`, `TEACHER`, or `ADMIN`.
3. Click **"Save Account"**. The account is created with status `Active`.

Managing Existing Accounts

- **Deactivation / Suspension**: Click *Deactivate* to temporarily revoke access. The user is blocked from logging in, and any active tokens will be rejected immediately on their next request.
- **Password Reset**: Click *Reset Password* to assign a temporary password to students who forgot their credentials.
- **Search & Filter**: Use the search bar to locate parishioners by name or email.

2.3 Curriculum Hierarchy Management

The curriculum studio at `/admin/curriculum` models the traditional Sunday School structure: **Grade** → **Subject** → **Chapter**.

Step 1: Grade Level Configuration

- Grades 1 through 12 correspond to standard educational levels.
- Each grade contains an English name (e.g. *Grade 1*), an Amharic liturgical name (e.g. *ክፍለ ፩*), and an `orderIndex`.
- The system strictly rejects grade levels outside the range of 1 to 12.

Step 2: Subject Definition

- Subjects belong to a specific grade level (e.g. *Church History*, *Dogmatics / ሃይማኖተ አበው*, *Liturgical Music / ዜማ*).
- Each subject must be assigned a unique subject code within its grade (e.g. `HIST-G1`, `DOGM-G1`).

Step 3: Chapter Definition

- Chapters are the fundamental study and download unit.
- Provide a chapter number (e.g. `1`), an English title, an Amharic title, and a study description.
- Created chapters remain in **Draft (Unpublished)** status until full materials are configured.

2.4 Textbook Document & Clamped Page Range Mapping

Every chapter requires an official textbook PDF document. To preserve student focus and prevent cognitive overload, the platform clamps the reading view to only the assigned chapter pages:

Textbook Configuration Procedure

1. From `/admin/curriculum`, select the chapter and navigate to the **Textbook PDF** tab.
2. Provide the Cloudinary or CDN raw PDF URL (e.g. `https://res.cloudinary.com/eotc/raw/upload/v1/grade1-history.pdf`).
3. Specify the total file size in bytes and provide the SHA-256 checksum for download verification.
4. **Configure Page Clamping**:
    - `startPage`: The first page of the chapter in the complete book (e.g., `15`).
    - `endPage`: The concluding page of the chapter (e.g., `35`).
    - `totalPages`: The total page count of the entire PDF file (e.g., `180`).
5. Click **"Save Textbook Mapping"**.

> **Page Clamping Validation Invariants**
>
> The API and Zod schemas enforce strict mathematical clamping: `startPage > 0`, `startPage ≤ endPage`, and `endPage ≤ totalPages`. Submitting an end page exceeding the book's total pages or a start page greater than the end page is rejected with `400 VALIDATION_ERROR`.

2.5 Authoring Practice Quizzes & Questions

Practice quizzes reinforce learning after textbook study. Quizzes are authored directly inside the chapter studio:

1. Navigate to the **Quizzes** tab and click **"Create Quiz"**.
2. Enter the quiz title (e.g. *ምዕራፍ ፩ መመዘኛ ጥያቄዎች*), time limit in minutes (e.g. `15`), and minimum passing score percentage (default: `70%`).
3. Add multiple-choice questions:
    - **Question Prompt**: Enter both English and Amharic text (e.g., *በዓለ ጎምሳ ሐዋርያት መንፈስ ቅዱስን የተቀበሉት የት ነበር?*).
    - **Point Value**: Assign positive point weights (e.g. `10` points per question).
    - **Choices**: Add at least 2 choices. Designate exactly one choice as **Correct** (`isCorrect: true`).
    - **Pedagogical Explanation**: Provide a detailed Amharic explanation that will be revealed to the student upon answering (e.g., *ሐዋርያት በኢየሩሳሌም በጽርሐ ጽዮን ሳሉ መንፈስ ቅዱስ ወረደባቸው።*).
4. Click **"Save Question"**.

2.6 Attaching Approved Reference Documents & Links

Under the **References** tab, administrators can supply supplementary readings:

- **Downloadable Reference PDFs**: Check "*Is Downloadable*" and provide a Cloudinary PDF link with file size and checksum. This document will be downloaded and saved to the student's device for full offline reading.
- **External Web Resources**: Leave "*Is Downloadable*" unchecked for external articles or video links. The student apps will clearly label these as "*Requires Internet*".

2.7 Atomic Package Publishing & Integrity Checksums

Publishing locks the chapter and compiles the canonical **Learning Package Manifest**.

> **Publishing Preconditions**
>
> The system verifies all 4 criteria before allowing publication:
>
> 1. A textbook PDF with valid clamped page ranges is configured.
> 2. At least one practice quiz is attached.
> 3. The quiz contains at least one question.
> 4. Every question has at least one valid correct choice.

When the administrator clicks **"Publish Chapter"**, the API computes a SHA-256 checksum over the entire package contents, sets `isPublished: true`, and increments the chapter version. The package is now immediately available for student offline downloads.

3. Student Manual ("How to Use — Student")

3.1 Web PWA & Mobile App Access and Installation

Students can access their curriculum through either a web browser on laptops/desktops or the dedicated mobile application on Android and iOS devices.

Installing the Web PWA (Progressive Web App)

1. Open Google Chrome, Microsoft Edge, or Safari and navigate to `https://learn.senbet.eotc-andnet.org`.
2. Click the **"Install App"** prompt in the address bar (or select "*Add to Home Screen*" on iOS Safari).
3. The EOTC Learning Platform icon appears on your home screen or desktop, running in a standalone window with zero browser chrome.

Installing the Mobile App

- **Parish Distribution APK**: Download the standalone Android APK directly from your parish coordinator and tap to install. No Google Play Store account is required.
- **App Store Releases**: Search for "*EOTC Senbet Learning*" in Google Play or Apple App Store and install.

3.2 Offline Authentication Credential Caching

The student enters their parish email and password while connected to Wi-Fi or cellular network:

- Upon successful login, the application securely caches user profile information, role credentials, and session tokens locally (in **IndexedDB** on Web, and **SQLite** on Mobile).
- **Airplane Mode Readiness**: The student can close the application completely, disconnect from the internet, and reopen the app hours or days later. The app immediately recognizes the local session, allowing full access to all previously downloaded packages without a network prompt.

3.3 Curriculum Discovery & Download Manager

The home screen displays the student's assigned Grade level with all associated subjects and chapters:

BADGE INDICATOR	COLOR	MEANING & OPERATIONAL GUIDANCE
`AVAILABLE_OFFLINE`	Green	Package is 100% downloaded, verified, and ready for study in airplane mode.
`NOT_DOWNLOADED`	Gray	Chapter is published online but has not been saved to this device yet. Requires network to download.
`DOWNLOADING`	Blue / Animated	Assets are actively transferring and checksums are being calculated. Do not close app.
`FAILED`	Red	Download interrupted or checksum mismatch. Click "*Retry Download*" to resume.

Downloading a Package for Offline Study

1. Click on the desired chapter.
2. Click the gold button labeled **"Download for Offline Study"**.
3. The Download Manager retrieves the manifest, downloads the clamped textbook PDF, downloads offline references, writes quizzes to local storage, and verifies all SHA-256 checksums.
4. Once complete, the badge flips to green **"Available Offline"**. You can now disconnect from the internet!

3.4 Offline Study Experience (Clamped PDF Reader & References)

When studying without internet connectivity, the student experience is completely seamless:

Reading Clamped Textbooks

- Open the chapter and click the **"Textbook"** tab.
- The PDF reader renders directly from the local device storage (IndexedDB Blob on Web; internal SQLite file storage on Mobile).
- **Strict Clamping**: Navigation is restricted exclusively to the chapter's assigned range (e.g. Page 15 to 35). The reader indicates: "*Page 1 of 20 (Book Page 15 of 180)*".
- **Automatic Reading Progress**: As the student advances pages, their current reading position is saved locally. Reaching the final assigned page automatically marks the chapter as completed.

Opening References

- Downloadable reference documents open instantly in the offline reader.
- External website links display an informative dialog: "*This external resource requires internet access. Please reconnect when available.*"

3.5 Taking Practice Quizzes in Airplane Mode

Students can test their comprehension anywhere — on a church bus, at home, or in rural areas:

1. Navigate to the **"Practice Quiz"** tab.
2. All questions and Amharic choices load instantaneously from local storage.
3. Select answers for each question.
4. Click **"Submit Practice Quiz"**.
5. **Immediate Offline Feedback**:
    - The local evaluation engine calculates the score immediately against the packaged answer keys.
    - The screen displays total score percentage, pass/fail status, and full Amharic explanations for every question.
6. **Durable Outbox Queue**:
    - The attempt is assigned a globally unique client attempt UUID (e.g. `ca000000-0000-4000-8000-000000000001`).
    - The attempt record is saved to the local outbox queue with status `PENDING_SYNC`.

3.6 Automatic Background Progress & Attempt Synchronization

The student never needs to manually trigger a sync button. The synchronization engine operates autonomously:

- **Connectivity Listener**: When Wi-Fi or cellular network returns, the application detects the network transition.

- **Batch Outbox Flush**: The client sends all pending quiz attempts and reading progress updates in an atomic payload to `POST /api/student/sync/batch`.
- **Server Scoring Truth**: The server recalculates question scores using the official database choices, preventing any score tampering.
- **Idempotent Deduplication**: If network drops mid-sync and the client retries, the server recognizes previously received attempt UUIDs and responds with status `DUPLICATE` without double-counting records.
- **Last-Write-Wins (LWW) Progress Resolution**: If progress is submitted from multiple devices out of order, the server compares `lastStudiedAt` timestamps, ensuring older device updates never regress a student's highest completed page.

3.7 Storage Management & Cleanup Controls

Students on devices with limited storage can manage their disk space at `/download-manager`:

- The dashboard displays exact local storage consumed (e.g. *42.8 MB used across 3 chapters*).
- Students can click **"Delete Local Files"** on any completed chapter to free device storage.
- Deleting local files does not erase completed quiz attempts or server-synchronized progress.

4. DevOps & Production Deployment Guide ("How to Deploy")

4.1 Infrastructure Requirements & Hardware Sizing

The platform is lightweight, compiled to optimized production artifacts, and containerized for deployment on any standard cloud provider (AWS, DigitalOcean, Hetzner, Linode) or on-premise parish server:

RESOURCE	MINIMUM SPECIFICATION	RECOMMENDED PRODUCTION	PARISH / LOCAL LAB SERVER
Operating System	Ubuntu 22.04 LTS / Debian 12	Ubuntu 24.04 LTS	Windows 11 / Ubuntu Server
CPU / Processor	2 vCPU (x86_64 or ARM64)	4 vCPU	2 Cores
System RAM	4 GB RAM	8 GB RAM	4 GB RAM
Disk Storage	30 GB SSD	80 GB NVMe with daily backups	50 GB SSD
Network Ports	80 (HTTP), 443 (HTTPS), 3000 (API internal), 3001 (Web internal), 5432 (Postgres internal)		

4.2 Environment Variables & Cryptographic Secrets

Create a production `.env` file in the repository root directory before deployment:

```
# ==============================================================================
# EOTC LEARNING PLATFORM — PRODUCTION ENVIRONMENT CONFIGURATION
# ==============================================================================

# Core Runtime Environment
NODE_ENV=production
PORT=3000
API_URL=https://api.senbet.eotc-andnet.org
WEB_URL=https://learn.senbet.eotc-andnet.org

# Database Connection (PostgreSQL 16)
DATABASE_URL="postgresql://postgres:YourSecurePassword2026!@postgres:5432/eotc_learning?schema=public&connection_limit=25&pool_timeout=15"

# Cryptographic Authentication Secrets (Generate with: openssl rand -hex 64)
JWT_ACCESS_SECRET="9a4b8c...64charHexSecret"
JWT_REFRESH_SECRET="f1e2d3...64charHexSecret"
JWT_ACCESS_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="30d"

# Cloudinary Storage Configuration (Textbooks & Downloadable References)
CLOUDINARY_CLOUD_NAME="eotc-andnet"
CLOUDINARY_API_KEY="your-api-key"
CLOUDINARY_API_SECRET="your-api-secret"
CLOUDINARY_FOLDER="eotc-learning"

# Client CORS Origins (Comma-separated)
CORS_ORIGIN="https://learn.senbet.eotc-andnet.org,https://admin.senbet.eotc-andnet.org"
```

> **Generating Production Cryptographic Keys**
>
> Run this command in terminal to generate secure, unguessable 64-byte secrets:
>
> ```
> node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
> ```

4.3 Database Setup, Migrations & Automated Seeding

Before starting services, deploy canonical database migrations and populate foundational Sunday School grades:

```
# 1. Start the PostgreSQL 16 container
docker compose up -d postgres

# 2. Apply Prisma database schema migrations
pnpm --filter "@eotc/api" prisma migrate deploy

# 3. Seed initial roles, default administrator, and Grades 1-4 curriculum
pnpm --filter "@eotc/api" prisma db seed
```

4.4 Containerized Deployment (Docker Compose + Nginx Gateway)

The recommended deployment method encapsulates all services in isolated, multi-stage Docker containers with an automated Nginx reverse proxy gateway:

Step 1: Inspect Multi-Container Architecture (docker-compose.prod.yml)

- `eotc_prod_postgres`: PostgreSQL 16 Alpine container with persistent data volume and automated `pg_isready` health checking.
- `eotc_prod_api`: Express.js API container built with multi-stage Alpine image, non-root user execution, and `dumb-init` signal forwarding.
- `eotc_prod_web`: Next.js 15 PWA container running optimized standalone production server on port 3001.
- `eotc_prod_nginx`: Production gateway managing reverse proxy routing, SSL termination, and caching rules.

Step 2: Nginx Gateway Caching & Security Rules (nginx/nginx.conf)

The gateway implements crucial PWA and performance rules:

- **Service Worker Cache Busting**: `/sw.js` is served with strict `Cache-Control: "no-cache, no-store, must-revalidate"` so students always receive the latest offline service worker instantly.
- **Web Manifest Caching**: `/manifest.json` is cached for 1 hour.
- **Immutable Static Assets**: `/_next/static/` files are cached for 1 year with `immutable` flag.
- **Security Headers**: Injects `X-Frame-Options SAMEORIGIN`, `X-Content-Type-Options nosniff`, and `Referrer-Policy strict-origin-when-cross-origin`.
- **Gzip Compression**: Compresses text, JSON, and application scripts on the fly.

Step 3: Launch Production Stack

```
# Build and launch all services in background
docker compose -f docker-compose.prod.yml up -d --build

# Verify container health and status
docker compose -f docker-compose.prod.yml ps
```

4.5 Host-Level Service Deployment (Systemd + PM2 + Node.js)

For bare-metal Linux servers or virtual machines deploying without Docker:

Option A: Linux Systemd Units

Create `/etc/systemd/system/eotc-api.service`:

```
[Unit]
Description=EOTC Learning Platform API Service
After=network.target postgresql.service

[Service]
Type=simple
User=eotc
WorkingDirectory=/var/www/senbet-learning/apps/api
EnvironmentFile=/var/www/senbet-learning/.env
ExecStart=/usr/bin/node dist/server.js
Restart=always
RestartSec=5
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=eotc-api

[Install]
WantedBy=multi-user.target
```

Option B: PM2 Process Manager

```
# Install PM2 globally
npm install -g pm2

# Build all monorepo apps
pnpm build

# Start API and Web services with cluster mode
pm2 start apps/api/dist/server.js --name "eotc-api" -i max
pm2 start "pnpm --filter '@eotc/web' start" --name "eotc-web"

# Save process list for system reboot persistence
pm2 save
pm2 startup
```

4.6 Mobile Application Build & Parish Offline Sideloading (EAS / APK)

The mobile application can be distributed directly to Sunday School students and teachers without requiring Google Play or Apple App Store accounts:

1. Direct Parish Offline APK Build (Android)

Using Expo Application Services (EAS), build an installable APK for parish distribution:

```
# Install EAS CLI globally
npm install -g eas-cli

# Navigate to mobile project
cd apps/mobile

# Build standalone Android APK
eas build --platform android --profile preview-apk --local
```

The generated `.apk` file can be copied to USB drives or local SD cards and installed on parish tablets or Android smartphones via USB transfer or local Wi-Fi sharing.

2. App Store Distribution Builds

```
# Build Google Play Android App Bundle (AAB)
eas build --platform android --profile production

# Build Apple App Store IPA
eas build --platform ios --profile production
```

4.7 Automated Database Backup & Disaster Recovery Scripts

Parish data integrity is mission-critical. Automated backup and restoration scripts are included in the repository:

Automated Backup Execution (scripts/backup-db.ps1)

```
# Execute automated database backup
powershell -ExecutionPolicy Bypass -File .\scripts\backup-db.ps1
```

This script dumps the PostgreSQL database into `backups/backup_YYYYMMDD_HHMMSS.sql.gz`, calculates SHA-256 integrity hashes, and automatically purges archives older than 30 days. On Linux servers, configure this command in `crontab` to run nightly at 2:00 AM.

Database Restoration (scripts/restore-db.ps1)

```
# Restore from backup archive
powershell -ExecutionPolicy Bypass -File .\scripts\restore-db.ps1 -BackupFile .\backups\backup_20261021_050000.s
ql.gz
```

4.8 Live Production Smoke Testing & Health Monitoring

Verify end-to-end service health immediately following any deployment using the automated smoke test script:

```
powershell -ExecutionPolicy Bypass -File .\scripts\smoke-test.ps1 -BaseUrl http://localhost:4000
```

Production Health Checklist

ENDPOINT	METHOD	EXPECTED STATUS	VERIFICATION GOAL
`/api/health`	`GET`	`200 OK`	API process uptime and PostgreSQL connection pool responsiveness.
`/api/curriculum/grades`	`GET`	`200 OK`	Public curriculum catalog discovery and database read operations.
`/api/admin/users`	`GET` (no token)	`401 Unauthorized`	Security check: verifies unauthenticated requests are strictly rejected.
`/sw.js`	`GET`	`200 OK`	Verifies offline Service Worker script is served with `no-cache` header.

5. Appendices & Reference Sheets

5.1 CLI Operations Cheat Sheet

TASK / OPERATION	COMMAND
Run All Monorepo Tests	`pnpm test`
Run Senior QA Integrity Suite	`pnpm --filter "@eotc/api" test qa-senior-e2e`
Run Typecheck Across Workspace	`pnpm typecheck`
Compile Production Builds	`pnpm build`
Deploy Prisma Migrations	`pnpm --filter "@eotc/api" prisma migrate deploy`
Seed Database Curriculum	`pnpm --filter "@eotc/api" prisma db seed`
Launch Docker Production Stack	`docker compose -f docker-compose.prod.yml up -d --build`
Execute Database Backup	`powershell -ExecutionPolicy Bypass -File .\scripts\backup-db.ps1`
Execute Live Smoke Tests	`powershell -ExecutionPolicy Bypass -File .\scripts\smoke-test.ps1 -BaseUrl http://localhost:4000`

5.2 API Error Code Reference Matrix

ERROR CODE	HTTP STATUS	ROOT CAUSE & CORRECTIVE ACTION
`INVALID_CREDENTIALS`	`401`	Incorrect email or password entered. Verify credentials or execute admin password reset.
`ACCOUNT_INACTIVE`	`403`	The account has been suspended by an administrator. Contact parish coordinator for reactivation.
`FORBIDDEN`	`403`	Access denied. Attempted to access an administrative endpoint with a student or teacher role.
`VALIDATION_ERROR`	`400`	Payload failed Zod validation (e.g. invalid email format, short password, or startPage > endPage).
`PRECONDITION_FAILED`	`400`	Cannot publish chapter: missing textbook PDF, zero quizzes attached, or questions missing choices.
`DUPLICATE_RESOURCE`	`409`	Attempted to create a duplicate grade level, email, or subject code within the same grade.
`NOT_FOUND`	`404`	Requested curriculum chapter, user, or endpoint does not exist.

5.3 Parish Offline Deployment Scenarios

Scenario A: Rural Parish Sunday School (Zero Internet)

1. The parish coordinator visits a central church or diocese with internet access once per term.
2. The coordinator downloads the latest curriculum packages onto a master laptop or USB drive using the Admin Portal.
3. At the rural church, the coordinator installs the standalone Android APK onto parish tablets or student smartphones via USB.
4. Students study textbooks, read reference documents, and take practice quizzes completely offline throughout the term.
5. When the coordinator returns to internet connectivity, the student devices batch-sync their accumulated quiz results in seconds.

Scenario B: Urban Parish Computer Lab (Local Network Deployment)

1. Deploy the Docker Compose stack onto a local parish lab PC running Windows 11 or Linux.
2. Connect lab computers or student tablets to the local parish Wi-Fi router (no internet uplink required).
3. Students navigate their browser to `http://parish-server-ip:3001` to access the Web PWA.
4. All student attempts and reading progress synchronize locally to the parish PostgreSQL server.